

旁, 先研究一下将 `operator*` 写成 `Rational` 成员函数的写法:

```
class Rational {
public:
    ...
    const Rational operator* (const Rational& rhs) const;
};
```

(如果你不确定为什么这个函数被声明为此种形式, 也就是为什么它返回一个 `const by-value` 结果但接受一个 `reference-to-const` 实参, 请参考条款 3, 20 和 21。)

这个设计使你能够将两个有理数以最轻松自在的方式相乘:

```
Rational oneEighth(1, 8);
Rational oneHalf(1, 2);
Rational result = oneHalf * oneEighth;    //很好
result = result * oneEighth;              //很好
```

但你还不满足。你希望支持混合式运算, 也就是拿 `Rationals` 和……嗯……例如 `ints` 相乘。毕竟很少有什么东西会比两个数值相乘更自然的了——即使是两个不同类型的数值。

然而当你尝试混合式算术, 你发现只有一半行得通:

```
result = oneHalf * 2;          //很好
result = 2 * oneHalf;          //错误!
```

这不是好兆头。乘法应该满足交换律, 不是吗?

当你以对应的函数形式重写上述两个式子, 问题所在便一目了然了:

```
result = oneHalf.operator*(2);    //很好
result = 2.operator*(oneHalf);    //错误!
```

是的, `oneHalf` 是一个内含 `operator*` 函数的 `class` 的对象, 所以编译器调用该函数。然而整数 2 并没有相应的 `class`, 也就没有 `operator*` 成员函数。编译器也会尝试寻找可被以下这般调用的 `non-member operator*` (也就是在命名空间内或在 `global` 作用域内):

```
result = operator*(2, oneHalf);    //错误!
```

但本例并不存在这样一个接受 `int` 和 `Rational` 作为参数的 `non-member operator*`, 因此查找失败。

再次看看先前成功的那个调用。注意其第二参数是整数 2，但 `Rational::operator*` 需要的实参却是个 `Rational` 对象。这里发生了什么事？为什么 2 在这里可被接受，在另一个调用中却不被接受？

因为这里发生了所谓隐式类型转换（`implicit type conversion`）。编译器知道你正在传递一个 `int`，而函数需要的是 `Rational`；但它也知道只要调用 `Rational` 构造函数并赋予你所提供的 `int`，就可以变出一个适当的 `Rational` 来。于是它就那样做了。换句话说此一调用动作在编译器眼中有点像这样：

```
const Rational temp(2);           //根据 2 建立一个暂时性的 Rational 对象。
result = oneHalf * temp;         //等同于 oneHalf.operator*(temp);
```

当然，只因为涉及 `non-explicit` 构造函数，编译器才会这样做。如果 `Rational` 构造函数是 `explicit`，以下语句没有一个可通过编译：

```
result = oneHalf * 2;             //错误！（在 explicit 构造函数的情况下）
                                   //无法将 2 转换为一个 Rational。
result = 2 * oneHalf;             //一样的错误，一样的问题。
```

这就很难让 `Rational` class 支持混合式算术运算了，不过至少上述两个句子的行为从此一致☺。

然而你的目标不仅在一致性，也要支持混合式算术运算，也就是希望有个设计能让以上语句通过编译。这把我们带回到上述两个语句，为什么即使 `Rational` 构造函数不是 `explicit`，仍然只有一个可通过编译，另一个不可以：

```
result = oneHalf * 2;             //没问题（在 non-explicit 构造函数的情况下）
result = 2 * oneHalf;             //错误！（甚至在 non-explicit 构造函数的情况下）
```

结论是，只有当参数被列于参数列（*parameter list*）内，这个参数才是隐式类型转换的合格参与者。地位相当于“被调用之成员函数所隶属的那个对象”——即 `this` 对象——的那个隐喻参数，绝不是隐式转换的合格参与者。这就是为什么上述第一次调用可通过编译，第二次调用则否，因为第一次调用伴随一个放在参数列内的参数，第二次调用则否。

然而你一定也会想要支持混合式算术运算。可行之道终于拨云见日：让 `operator*` 成为一个 `non-member` 函数，俾允许编译器在每一个实参身上执行隐式类型转换：

```

class Rational {
    ... //不包括 operator*
};

const Rational operator*(const Rational& lhs, //现在成了一个
                        const Rational& rhs) //non-member 函数
{
    return Rational(lhs.numerator() * rhs.numerator(),
                    lhs.denominator() * rhs.denominator());
}

Rational oneFourth(1, 4);
Rational result;
result = oneFourth * 2; //没问题
result = 2 * oneFourth; //万岁, 通过编译了!

```

这当然是个快乐的结局, 不过还有一点必须操心: `operator*` 是否应该成为 `Rational` class 的一个 `friend` 函数呢?

就本例而言答案是否定的, 因为 `operator*` 可以完全藉由 `Rational` 的 `public` 接口完成任务, 上面代码已表明此种做法。这导出一个重要的观察: `member` 函数的反面是 `non-member` 函数, 不是 `friend` 函数。太多 C++ 程序员假设, 如果一个“与某 class 相关”的函数不该成为一个 `member` (也许由于其所有实参都需要类型转换, 例如先前的 `Rational` 的 `operator*` 函数), 就该是个 `friend`。本例表明这样的理由过于牵强。无论何时如果你可以避免 `friend` 函数就该避免, 因为就像真实世界一样, 朋友带来的麻烦往往多过其价值。当然有时候 `friend` 有其正当性, 但这个事实依然存在: 不能够只因函数不该成为 `member`, 就自动让它成为 `friend`。

本条款内含真理, 但却不是全部的真理。当你从 `Object-Oriented C++` 跨进 `Template C++` (见条款 1) 并让 `Rational` 成为一个 `class template` 而非 `class`, 又有一些需要考虑的新争议、新解法、以及一些令人惊讶的设计牵连。这些争议、解法和设计牵连形成了条款 46。

请记住

- 如果你需要为某个函数的所有参数 (包括被 `this` 指针所指的那个隐喻参数) 进行类型转换, 那么这个函数必须是个 `non-member`。

条款 25：考虑写出一个不抛异常的 swap 函数

Consider support for a non-throwing swap.

swap 是个有趣的函数。原本它只是 STL 的一部分，而后成为异常安全性编程（exception-safe programming，见条款 29）的脊柱，以及用来处理自我赋值可能性（见条款 11）的一个常见机制。由于 swap 如此有用，适当的实现很重要。然而在非凡的重要性之外它也带来了非凡的复杂度。本条款探讨这些复杂度及因应之道。

所谓 *swap*（置换）两对象值，意思是将两对象的值彼此赋予对方。缺省情况下 *swap* 动作可由标准程序库提供的 swap 算法完成。其典型实现完全如你所预期：

```
namespace std {
    template<typename T>           //std::swap 的典型实现;
    void swap( T& a, T& b)         //置换 a 和 b 的值.
    {
        T temp(a);
        a = b;
        b = temp;
    }
}
```

只要类型 *T* 支持 *copying*（通过 *copy* 构造函数和 *copy assignment* 操作符完成），缺省的 swap 实现代码就会帮你置换类型为 *T* 的对象，你不需要为此另外再做任何工作。

这缺省的 swap 实现版本十分平淡，无法刺激你的肾上腺。它涉及三个对象的复制：*a* 复制到 *temp*，*b* 复制到 *a*，以及 *temp* 复制到 *b*。但是对某些类型而言，这些复制动作无一必要；对它们而言 swap 缺省行为等于是把高速铁路铺设在慢速小巷弄内。

其中最主要的就是“以指针指向一个对象，内含真正数据”那种类型。这种设计的常见表现形式是所谓“pimpl 手法”（pimpl 是 “pointer to implementation” 的缩写，见条款 31）。如果以这种手法设计 *Widget class*，看起来会像这样：

```
class WidgetImpl {                //针对 Widget 数据而设计的 class;
public:                           //细节不重要。
    ...
private:
    int a, b, c;                  //可能有许多数据,
    std::vector<double> v;        //意味复制时间很长。
    ...
};
```